REPORT PROJECT 4

Mancala problem :

Task 1 : What is the utility function of the algorithm ?

The utility function is the minmax function below, it is used to evaluate how good or bad is a specific move, with a score :

```
def minmax(board, nxt_turn, playerTurnIn, depth, me): #me represent
wich player i am

    if nxt_turn == 1:
        options = np.where(np.array(board[0:6]) > 0)[0]
        options = options[options != 6]

    elif nxt_turn == 2:
        options = np.where(np.array(board[7:13]) > 0)[0]
        options = options[options != 13]

    if depth == 0 or (sum(board[:6]) == 0) or (sum(board[7:13]) == 0)
or len(options) == 0: #base case
            return  countScorePlayer1(board) if me == 1 else
-countScorePlayer1(board)

    if nxt_turn == playerTurnIn:
            best_val = float('-inf')
            for move_ind in options:
                move = move_ind + 1
                board_2, nxt_turn_2 = play(nxt_turn, move, board[:])
                eval = minmax(board_2, nxt_turn_2, nxt_turn, depth -
1, me)
                best_val = max(best_val, eval)
    else:
            best_val = float('inf')
            for move_ind in options:
                move = move_ind + 1
                board_2, nxt_turn_2 = play(nxt_turn, move, board[:])
                eval = minmax(board_2, nxt_turn_2, nxt_turn, depth -
1, me)
                best_val = min(best_val, eval)
    return best_val
```

Let's analyse this function :

```
if nxt_turn == 1:
        options = np.where(np.array(board[0:6]) > 0)[0]
        options = options[options != 6]

    elif nxt_turn == 2:
        options = np.where(np.array(board[7:13]) > 0)[0]
        options = options[options != 13]
```

This part of the code finds all the possible moves the player (next_turn) can make.

```
if depth == 0 or (sum(board[:6]) == 0) or (sum(board[7:13]) == 0) or
len(options) == 0: #base case
            return  countScorePlayer1(board) if me == 1 else
-countScorePlayer1(board)
```

This is the base case, it is applied when `depth == 0` meaning when we have already gone deep enough in the tree, and are ready to make a decision based on all the information we collected, without simulating anymore moves. It is also applied when `(sum(board[:6]) == 0) or (sum(board[7:13]) == 0)` which means that the game has ended because the sum of the holes of a player (except his storage) is empty, or when `len(options) == 0` meaning the player has no legal move.
This return `countScorePlayer1(board) if me == 1 else -countScorePlayer1(board)`, meaning countScorePlayer1 if the player the AI is playing (represented by the variable `me`) is `Player1`, and `-countScorePlayer1(board)` if the player the AI is playing is `Player2`, because that is the player 2 score since the function is :

```
def countScorePlayer1(boardGame):
    (p1s, p2s) = countPoints(boardGame)
    return int(p1s - p2s)


def countPoints(boardGame):
    return (boardGame[6], boardGame[13])
```

Finally,

```
if nxt_turn == playerTurnIn:
            best_val = float('-inf')
            for move_ind in options:
                move = move_ind + 1
                board_2, nxt_turn_2 = play(nxt_turn, move, board[:])
                eval = minmax(board_2, nxt_turn_2, nxt_turn, depth -
1, me)
                best_val = max(best_val, eval)
    else:
            best_val = float('inf')
            for move_ind in options:
                move = move_ind + 1
                board_2, nxt_turn_2 = play(nxt_turn, move, board[:])
                eval = minmax(board_2, nxt_turn_2, nxt_turn, depth -
1, me)
                best_val = min(best_val, eval)
    return best_val
```

For the last part, this is where the recursion occurs. if `nxt_turn == playerTurnIn` meaning if the same player that had just played, in the simulation, is the one that is going to play again now, then we set `best_val = float('-inf')` since we want to maximize the value. Then for every possible move the player can make, we simulate it with the play function and get back a copy of the resulted board and whose turn it is after that move, with those information we call, recursively, our utility function `minmax`, with the copy of the board after the move, the next player to play, the player that had just played, `depth - 1,` since we just simulated a move, and the AI's player number. we update `best_val = max(best_val, eval)` everytime we find a higher value than with the moves before. When `nxt_turn != playerTurnIn,` we want to minimize the other player's score, so the process is the same except `best_val = float('inf')` and we update it everytime we find a lower value `best_val = min(best_val, eval)`.